

I have a problem statement I want to research about.

Here's a workflow that I currently have:

1. I am coding and asking Claude to write documents, it writes an MD file, I share the md file wiht the teammate, and he has to open an MD editor (VS code) to be able to read the non raw markdown text.

2. If I want Claude to write a doc that is not in md, it currently writes code using pandoc, transforming md to xml etc. Many tool calls, many skills needed.

I want to research whether there are any solutions for a word like markdown editor. Here's what I have found so far:

- Haxiom from this reddit post:

https://www.reddit.com/r/Markdown/comments/1qbbku8/author_markdown_share_and_collab_in_enterprise/

- obsedian (works but not collaborative, very limited features).

Research exisitng solutions and posts talking about the pain

Research completed in 10m · 42 citations · 247 searches

Collaborative, Word-Like Workflows for Markdown Docs



Collaborative, Word-Like Workflows for Markdown Docs

Problem framing and success criteria

Our workflow highlights two related frictions that show up whenever Markdown leaves “developer land”:

1. **Reading friction:** a teammate receives a `.md` file and sees raw Markdown unless they open a preview-capable tool (e.g., a code editor, a specialized Markdown app, or a web platform that renders Markdown). This is especially common because many default file viewers and browsers treat `.md` as plain text unless something explicitly renders it (extensions or web platforms).¹
2. **Format-conversion friction:** if the output must be “Word-like” (WYSIWYG editing, comments, trackable edits, familiar sharing), Markdown-to-Word/PDF pipelines often end up as **Pandoc + templates + glue code**, which is powerful but operationally heavy and easy to get “almost right” rather than



that's a lot of good information.

what do you think about this point:

2. If I want Claude to write a doc that is not in md, it currently writes code using pandoc, transforming md to xml etc. Many tool calls, many skills needed.

This is a big issue no?

Yeah — this is a **real problem**, and more importantly, it's a *signal of a broken abstraction*.

Let me be blunt:

You're forcing an LLM to behave like a document conversion pipeline instead of a

document editor.

What's actually going wrong

Your current flow:

Claude → Markdown → Pandoc → XML/Doc/etc → teammate reads

This has 3 fundamental issues:

1. Wrong interface layer

You're interacting with **documents as files**, not as **rendered objects**.

- Markdown = *storage format*
- What you want = *presentation layer* (like Microsoft Word or Google Docs)

You're basically asking:

"Can Claude output something readable?"

Instead of:

"Can I view/edit what Claude wrote in a native format?"

2. You're using Pandoc as glue (bad sign)

Pandoc is powerful — but when it becomes part of your *daily workflow*, that's friction.

Signals you're overcomplicating:

- multiple tool calls
- format conversions
- dependency on CLI tooling
- teammates needing dev tools to read docs

That's not scalable beyond you.

3. LLMs are being used inefficiently

Right now:

- Claude generates Markdown
- Then you *post-process it*

But LLMs are perfectly capable of:

- generating structured docs directly
- or writing into a rich editor via API

You're missing that layer.

Is this a big issue?

Yes — but not for the reason you think.

It's not just "too many steps."

It's this:

You don't have a native interface between LLM output and human-readable documents.

That's the real gap.

What the *correct* architecture looks like

You want:

Claude → rich doc (live, editable, shareable)



NOT:

Claude → md → pandoc → docx → open somewhere



Existing solutions that partially solve this

1. Notion (closest to what you want)

- LLM writes directly into rich blocks
- no Markdown exposure needed

- collaborative by default

Problem: not dev-native enough

2. Coda

- programmable docs
- can integrate AI + structured output

Problem: heavy, not dev workflow friendly

3. Typora

- WYSIWYG Markdown (no raw MD)
- feels like Word

Problem: no real collaboration

4. HackMD

- real-time collaboration
- rendered Markdown

Problem: still Markdown-first mentally

5. Google Docs + API

- LLM writes directly into doc via API

This is actually underrated and very viable.

The deeper insight (this is the important part)

What you're hitting is a **category gap**:

There is no dominant "LLM-native document editor" yet.

Current tools are:

- Markdown-first (dev tools)

- or Rich-doc-first (non-dev tools)

But not:

LLM-first + dev-friendly + collaborative + rich UI

My take (pushback)

You're solving this the hard way.

You don't need better Markdown tools.

You need to stop thinking:

"How do I convert Markdown into readable docs?"

And instead ask:

"Where should Claude write so no conversion is needed?"

What I would do (practical)

Short-term:

- Pipe Claude output directly into:
 - Notion page
 - or Google Docs via API

Kill Pandoc entirely.

Mid-term (if you're serious / startup angle):

- Build:
 - markdown-compatible backend
 - rich-text collaborative frontend
 - LLM writes directly into blocks

Basically:

"Figma for docs + Claude inside"

Bottom line

Yes, it's a big issue — but it's not a tooling problem.

It's a **missing product layer**.

And honestly, this is exactly the kind of friction that turns into a real product if you lean into it.



so notion is the answer? Like why does notion not solve this and does the word approach work for this? Why/why not

Short answer: **Notion helps, but it's not *the* answer. Word is worse for your use case.**

Let's break it down cleanly.

Why Notion *feels* like the answer

Notion solves a few of your exact pain points:

-  **No format conversion** (no Pandoc nonsense)
-  **LLM writes directly into a readable doc**
-  **Block-based structure → easier for AI to reason about**
-  **Collaboration is native**

So compared to your current pipeline:

It removes like 80% of the friction immediately.

But here's where Notion breaks (important)

If you're building something serious (which you are), Notion has real limitations:

1. Weak "programmability"

- You can't precisely control structure
- Hard to enforce schemas (e.g., "this section must be X")
- API is limited / clunky

👉 For you (ML/dev mindset), this will feel **constraining fast**

2. AI is bolted on, not native

- Notion AI = prompt → blob of text
- Not:
 - structured outputs
 - tool-using agents
 - iterative workflows

👉 You lose the ability to build **multi-step intelligent pipelines**

3. Poor integration with dev workflows

- Doesn't plug cleanly into:
 - your codebase
 - your pipelines
 - versioning

👉 It becomes a **side tool**, not part of your system

4. Scaling problem (this matters more than people think)

- Works great for:
 - notes
 - docs
 - Breaks for:
 - programmatic doc generation
 - large structured outputs
 - automated pipelines
-

What about Microsoft Word?

Word looks like the "final form" (nice docs), but it's actually worse for your setup.

Where Word *is* good

-  Final polished documents
-  Formatting control (styles, layout)
-  Industry standard for sharing

Why Word fails for your use case

1. Zero LLM-native workflow

- No clean way for Claude to “write into Word”
- You’re back to:
 - generate → convert → import

 same Pandoc problem, just dressed differently

2. Not programmatic

- Hard to:
 - automate structure
 - enforce schemas
 - build pipelines

 Word is **human-first, not system-first**

3. Collaboration is clunky

Compared to modern tools:

- versioning sucks
- comments are heavy
- no real-time intelligence

The real comparison (this is what you care about)

Capability	Notion	Word
LLM-native writing	 decent	 bad

Capability	Notion	Word
No conversion needed	✅	❌
Programmability	❌ weak	❌ very weak
Dev integration	❌	❌
Final polish	🟡	✅
Automation pipelines	❌	❌

So what's the honest answer?

Notion is a band-aid, not a foundation

Use it if:

- you want to move fast
- you want to remove Pandoc today

Don't rely on it if:

- you want a real system
- you care about automation + structure

What actually works (principled answer)

You need **3 layers**, not 1 tool:

1. Generation layer

- Claude / LLM
- outputs **structured data**, not just text

2. Representation layer (THIS is what you're missing)

Instead of Markdown:

- JSON schema
- block-based model

- AST-like structure

Example:

```
{
  "title": "...",
  "sections": [
    {"type": "paragraph", "content": "..."},
    {"type": "table", "data": [...]}
  ]
}
```

3. Rendering layer

- Notion / web UI / Google Docs / export to Word
-

The key shift (don't miss this)

Right now you're thinking:

"LLM → format → document"

You should be thinking:

"LLM → structured representation → render anywhere"

My blunt recommendation

If you want speed:

- Use Notion
- kill Pandoc
- move on

If you want leverage (and this fits your profile way more):

- build a **structured doc pipeline**
- render to:
 - Notion (optional)

- web UI
 - export to Word when needed
-

Bottom line

- Notion = good *temporary fix*
 - Word = dead end for pipelines
 - Your real problem = **you're missing a structured middle layer**
-

If you want, I can sketch what a **clean architecture for your exact use case** would look like (with APIs + minimal infra).



but the structured layer is exactly what chatGPT/Claude can do... They create code using jsons etc. then run scripts etc. to create doc

Yes — and that's exactly why your point is strong.

The structured layer is **possible today** because ChatGPT/Claude can already do:

- emit JSON
- call tools
- generate intermediate representations
- run scripts to render output

So the issue is **not capability**.

The issue is **product ergonomics**.

The distinction that matters

There are two very different things:

1. "Can the model do it?"

Yes.

A model can absolutely do:

```
{
  "title": "Design Doc",
  "sections": [
    {"type": "heading", "level": 1, "text": "Problem"},
    {"type": "paragraph", "text": "We need..."}
  ]
}
```

then call:

- a renderer
- a doc API
- a conversion script
- a formatter

That part is real.

2. "Does the user have a clean native workflow for it?"

Usually no.

That is where the pain is.

Because right now the workflow is still:

- prompt the model
- hope it emits the right structure
- hope the tool call works
- hope the renderer preserves formatting
- hope collaboration still works
- hope edits round-trip cleanly

That is not the same as having a product where:

"Claude writes a doc and my teammate just opens it."

So your claim is right, but incomplete

You're basically saying:

"Why do we need a separate structured layer if the LLM can already generate one

dynamically?"

Answer:

Because **dynamic generation is not the same as persistent document infrastructure.**

The model can generate structure, but you still need a system that handles:

- persistence
- rendering
- collaborative editing
- permissions
- comments
- version history
- export/import
- round-tripping after humans edit it

That is the missing part.

Why the current approach still feels bad

Because today the LLM is acting like an **improvised compiler pipeline.**

It is doing:

- authoring
- schema generation
- transformation
- script writing
- execution orchestration

all at once.

That works, but it is fragile.

Every extra step adds failure modes:

- malformed JSON
- script bugs
- style drift
- docx weirdness
- XML edge cases
- broken re-import after human edits

So yes, Claude can do it.

But needing Claude to constantly re-create the pipeline is exactly the smell.

The real product gap

The gap is not:

“LLMs can’t produce rich docs.”

They can.

The gap is:

“There is no dominant system where LLMs write into a stable document model that humans can also edit naturally.”

That’s why Notion sort of helps, but doesn’t fully solve it:

- it gives you persistence + collaboration
- but weak control / weak programmability

And Word doesn’t solve it:

- it gives you presentation
- but terrible automation ergonomics

Said differently

Your current system is:

LLM = author + compiler + converter + automation glue



What you want is:

LLM = author
Platform = document system



That separation is the big deal.

So is this a big issue?

Yes, but more precisely:

- It is **not** that models are incapable.
- It is that users are forced to build ad hoc doc pipelines around capable models.

That is annoying for power users.

It is fatal for normal teams.

The strongest version of your argument

A good way to phrase the problem statement is:

Modern LLMs can already generate structured representations and conversion code for rich documents, but turning that capability into a stable collaborative workflow still requires too many tool calls, too much glue code, and too much technical skill.

That's much sharper than just "markdown is annoying."

Bottom line

You're right that the structured layer can be generated by ChatGPT/Claude.

The problem is that today it is **generated ad hoc**, not **provided as a reliable collaborative substrate**.

That's the difference between:

- a clever workaround
- and a real product.

If you want, I can help you turn this into a tighter startup-style problem statement with: user pain, current workaround, why existing tools fail, and what the wedge is.



how does notion work in the backend?

Notion looks simple on the surface, but under the hood it's very different from Markdown/Word. The key idea:

Notion is not a document editor. It's a structured block database with a renderer.

Core mental model

Instead of:

Document = big string (Markdown / Word)



Notion is:

Document = tree of blocks (stored as structured data)



1. Block-based data model

Everything in Notion is a **block**:

- paragraph
- heading
- bullet list
- code block
- table
- image
- toggle
- database

And blocks are:

```
{
  "id": "block_id",
  "type": "paragraph",
  "content": [
    { "text": "Hello world", "annotations": { "bold": true } }
  ],
  "children": []
}
```

So a page is literally:

```
{
  "type": "page",
  "children": [block1, block2, block3]
}
```

👉 This is basically a **JSON AST** for documents

2. Storage layer (database, not files)

Notion does NOT store .md or .docx .

Instead:

- Blocks are rows in a database
- Each block has:
 - type
 - properties
 - parent_id
 - order

Think:

```
blocks table:  
- id  
- parent_id  
- type  
- content (JSON)  
- position
```



👉 This is why:

- edits are granular
- collaboration is smooth
- no merge conflicts like files

3. Real-time collaboration (CRDT-ish / OT hybrid)

Notion supports multiple users editing at once.

Under the hood:

- operations are **patches on blocks**, not full document rewrites
- something like:
 - insert block
 - update text
 - reorder blocks

Conceptually similar to:

- Operational Transformation
- Conflict-free Replicated Data Type

👉 That's why:

- no file locking
- no "version conflicts"
- instant sync

4. Rendering layer (this is key)

Notion separates:

Data (blocks) → Renderer → UI



The frontend:

- reads block JSON
- renders React components

Example:

```
{ type: "heading_1", text: "Hello" }  
→ <h1>Hello</h1>
```



👉 This is what you're missing in your workflow:

You're mixing **data + rendering + storage** into Markdown.

5. Rich text model

Inside blocks, text isn't plain:

```
{  
  "text": "Hello",  
  "annotations": {  
    "bold": true,  
    "italic": false,  
    "color": "red"  
  }  
}
```

So instead of Markdown like:

```
**Hello**
```

They store:

```
{ text: "Hello", bold: true }
```

👉 No parsing needed → cleaner + deterministic

6. Databases = first-class citizens

This is where Notion is actually powerful:

A "table" is not visual — it's real structured data:

```
{  
  "properties": {  
    "Name": "Task A",  
    "Status": "Done",  
    "Owner": "Igor"  
  }  
}
```

And views (table, kanban, calendar) are just **renderings of the same data**.

7. API layer

Notion exposes an API:

- create blocks
- update blocks
- query pages

But limitations:

- not super fast

- not real-time
- limited schema control

👉 That's why it feels "not programmable enough" for you

Why this matters for your problem

Compare your current pipeline:

LLM → Markdown → Pandoc → Doc



vs Notion's model:

LLM → Blocks (JSON) → Stored → Rendered instantly



No conversion step.

Why Notion still feels limiting

Even though the architecture is correct, it has constraints:

1. You don't control the schema

- you can't define strict document structures easily

2. Weak automation layer

- no native "agent pipelines"
- AI is just text generation, not structured workflows

3. API friction

- not designed for high-frequency programmatic writes
-

The important insight

Notion solved:

"How do we store and render documents without Markdown or files?"

But it did NOT solve:

"How do LLMs and humans co-create structured documents programmatically?"

If you abstract it cleanly

Notion architecture:

[Block JSON] → [Database] → [Realtime sync] → [Renderer]



Your current system:

[Markdown string] → [Conversion tools] → [Viewer]



What you should take away

If you're thinking like a builder:

- Markdown = **serialization format**
- Notion = **data model + UI system**

Your pain is coming from using:

a serialization format as your primary interface

Blunt takeaway

Notion's backend is actually the *correct direction*.

But:

- it's closed
- not programmable enough
- not LLM-native

If you want, I can sketch:

- a minimal "Notion-like backend" you could build

- specifically optimized for LLM-generated docs + collaboration
(which is honestly a very interesting space given what you're exploring)



so how does notion convert a md into the blocks?

Good question—and this is where people often misunderstand Notion.

Notion doesn't really "convert Markdown" in a deep sense.

It does a **shallow parse** → **block mapping**, not a full Markdown AST transformation like Pandoc.

What actually happens when you paste/import Markdown

Step 1: Line-by-line parsing (not full-document parsing)

Notion basically scans text like this:

```
# Title
Some paragraph text

- item 1
- item 2
```

And turns it into blocks:

```
[
  { "type": "heading_1", "text": "Title" },
  { "type": "paragraph", "text": "Some paragraph text" },
  { "type": "bulleted_list_item", "text": "item 1" },
  { "type": "bulleted_list_item", "text": "item 2" }
]
```

👉 This is **pattern matching**, not full semantic parsing.

Step 2: Inline formatting parsing

Inside each line, it parses simple Markdown:

```
**bold** *italic* `code`
```

→

```
[
  { "text": "bold", "bold": true },
  { "text": "italic", "italic": true },
  { "text": "code", "code": true }
]
```

Again:

- regex-level parsing
- not a full Markdown spec implementation

Step 3: Nesting (limited)

For things like:

```
- parent
  - child
```

It creates:

```
{
  "type": "bulleted_list_item",
  "text": "parent",
  "children": [
    { "type": "bulleted_list_item", "text": "child" }
  ]
}
```


But this breaks quickly for complex cases.

What Notion DOESN'T do (important)

It does NOT:

✗ Fully support Markdown spec

- tables → often broken or flattened
 - footnotes → ignored
 - complex nesting → inconsistent
 - HTML in Markdown → mostly ignored
-

✗ Preserve structure perfectly

Round-trip this:

Markdown → Notion → Markdown

You will lose:

- formatting fidelity
 - structure nuances
-

✗ Treat Markdown as source of truth

Markdown is just an **import convenience**, not a first-class format.

Internally:

Notion only cares about blocks.

Why this matters for your problem

You might be thinking:

“Why not just have Claude output Markdown and let Notion handle it?”

Because:

- Notion’s Markdown support is **lossy**
- It’s not deterministic
- It breaks for structured / programmatic docs

The deeper architectural point

There are two fundamentally different approaches:

Approach A (what you’re doing)

Markdown (string) → parse → render



Approach B (what Notion does)

Blocks (structured data) → render



Markdown is just a **bridge into B**, not part of the core system.

What a “better” pipeline would look like

Instead of:

Claude → Markdown → Notion parses → blocks



You could do:

Claude → blocks (JSON) → Notion API



Example:

```
{
  "parent": { "page_id": "...",
  "children": [
    {
```

```
    "object": "block",
    "type": "heading_1",
    "heading_1": {
      "rich_text": [{ "text": { "content": "Title" } }]
    }
  ]
}
```

👉 This skips:

- parsing ambiguity
- formatting loss
- conversion issues

Why Notion doesn't emphasize this

Because their target user is:

- non-technical
- copy/paste workflow

Not:

- LLM pipelines
- programmatic doc generation

Bottom line

- Notion's Markdown conversion is **lightweight and lossy**
- It's just:

“recognize patterns → map to blocks”

- Not:

“fully understand and preserve document structure”

The real takeaway for you

If you're serious about this problem:

Stop treating Markdown as the interface.

Either:

- write directly to Notion blocks via API
 - or
 - build your own block-based representation
-

If you want, I can show:

- a minimal function that takes Claude output → valid Notion block payload so you can completely eliminate Markdown from your workflow.



does notion have all the features as word? indentations, commenting, offline, etc.?

Short answer: **No — Notion does NOT have all the features of Word.**

It overlaps in some areas (especially collaboration), but they're built for *different jobs*.

Let's go point-by-point on the things you asked.

1. Indentation / formatting

Notion

- Supports:
 - basic indentation (nested blocks, lists)
 - headings, callouts, toggles
- BUT:
 - no fine-grained layout control
 - no pixel-perfect formatting
 - no advanced styles (like section breaks, margins, etc.)

Microsoft Word

- Extremely rich formatting:
 - precise indentation

- styles, spacing, margins
- page layout, columns, headers/footers

👉 Verdict:

Notion = **good enough for docs**

Word = **professional formatting engine**

2. Commenting / collaboration

Notion

- Comments on blocks
- @mentions
- real-time editing

Word

- Comments + track changes (huge difference)
- version history
- suggestion mode

👉 Key difference:

- Notion = "discussion"
- Word = **formal review system**

👉 Verdict:

Word is much stronger for **editing workflows (legal, academic, review-heavy)**

3. Offline support

Notion

- Weak offline
- Only cached pages work
- large pages/databases often break offline

Nuclino +1

Word

- Full offline support

- designed desktop-first

👉 Verdict:

Notion is basically **online-first**, Word is **offline-first**

4. Collaboration model (this is where Notion wins)

Notion

- real-time by default
- shared workspace
- feels like Google Docs + database

Word

- collaboration exists (via OneDrive)
- but historically more file-based

👉 Verdict:

Notion is better for **living team docs / wikis**

5. Structure vs layout (THIS is the real difference)

This is the most important one.

Notion

- structured blocks
- flexible but "loose"
- great for:
 - internal docs
 - knowledge bases
 - evolving content

Word

- layout-first
- rigid and polished
- great for:

- final deliverables
- PDFs
- contracts, reports

👉 Insight:

Notion = **system of knowledge**
Word = **finished artifact**

6. Feature coverage overall

Here’s the honest comparison:

Feature	Notion	Word
Basic formatting	✓	✓
Advanced formatting	✗	✓
Comments	✓	✓
Track changes	✗	✓
Real-time collaboration	✓	🟡
Offline editing	✗ weak	✓ strong
Databases / structured data	✓	✗
Templates	✓	✓
Print-ready docs	✗	✓

So why doesn’t Notion replace Word?

Because they optimize for different things:

Notion optimizes for:

- collaboration

- flexibility
- evolving documents
- internal workflows

Word optimizes for:

- precision
- formatting
- final outputs
- standardization

What this means for YOUR problem

This is the key takeaway:

Notion solves your Markdown → readability problem
but it does NOT replace Word as a “final document system”

So your pipeline becomes:

Better workflow

Claude → Notion (collaborate, edit, iterate)
→ export → Word/PDF (final)



NOT:

Claude → Markdown → Pandoc → Word



Blunt answer

- Notion is **not a full Word replacement**
- Word is **not a good system for your workflow**

You actually need both roles:

- Notion (or similar) → **working environment**
 - Word → **final output layer**
-

The deeper insight (don't miss this)

You're trying to find:

"One tool that does everything"

That tool doesn't exist.

Because:

- "editing system"
- and "publishing system"

are fundamentally different problems.

If you want, I can map out the **cleanest possible pipeline for your exact use case (LLM + teammates + docs)** that avoids both Markdown pain *and* Word friction.

    ...  Sources

can you explain how word works in the background

Good—this is the right question, because Word is basically the *opposite architecture* of Notion.

Word is a layout + formatting engine built on top of a document tree (XML), not a block database.

Core mental model

Instead of:

Document = blocks (Notion)



Word is closer to:

Document = formatted text stream + layout rules



1. File format: .docx = zipped XML

A Word file is literally:

.docx = ZIP file → contains XML files



Inside you'll find:

- document.xml → main content
- styles.xml → styles (Heading 1, Normal, etc.)
- numbering.xml → lists
- comments.xml → comments
- media (images)

Based on:

- Office Open XML

Example (simplified)

```
<w:document>
  <w:body>
    <w:p>
      <w:r>
        <w:t>Hello world</w:t>
      </w:r>
    </w:p>
  </w:body>
</w:document>
```

Where:

- w:p = paragraph
- w:r = run (piece of text)
- w:t = text

2. Text model: “runs” and formatting

Word doesn't store:

****bold****

Instead:

```
<w:r>
  <w:rPr><w:b/></w:rPr>
  <w:t>bold</w:t>
</w:r>
```

👉 Formatting is attached to **text segments**, not parsed from syntax.

3. Styles system (very important)

Word separates:

content vs style



Example:

- “Heading 1” is not just big text
- it's a named style with:
 - font size
 - spacing
 - hierarchy

This lives in `styles.xml`

👉 This is why Word can:

- auto-generate table of contents
 - enforce consistency
 - support enterprise docs
-

4. Layout engine (this is Word's superpower)

Word is fundamentally a **page layout engine**.

It computes:

- line wrapping
- page breaks
- margins
- columns
- headers/footers
- pagination

This is something Notion basically doesn't do.

5. Document structure (less explicit than Notion)

Word *does* have structure, but it's implicit:

- paragraphs
- sections
- lists
- tables

But:

- not cleanly hierarchical like blocks
 - more like a **flow with embedded objects**
-

6. Comments & track changes

Stored separately:

- `comments.xml`
- revision tags inside document

Example:

```
<w:ins>new text</w:ins>  
<w:del>old text</w:del>
```

👉 This enables:

- track changes
 - formal editing workflows
-

7. Rendering pipeline

Word does:

XML → layout engine → visual page



This is closer to:

- a browser rendering HTML/CSS
 - than a block renderer
-

8. Why Word is hard to automate

Because:

1. It's layout-first, not structure-first

- meaning depends on formatting
 - not clean data structures
-

2. XML is verbose and fragile

- hard to generate correctly
 - easy to break
-

3. No clean abstraction layer

- you manipulate:
 - raw XML
 - or high-level APIs (limited)
-

👉 That's why you ended up using:

- Pandoc

Compare to Notion (this is the key insight)

Aspect	Notion	Word
Storage	block JSON	XML
Model	structured tree	formatted flow
Rendering	simple	complex layout engine
Collaboration	native	layered on top
Programmability	medium	low
Output quality	medium	very high

Why Word doesn't fit your workflow

Your problem:

“Claude → readable, collaborative doc”

Word forces:

Claude → XML/docx → open in Word



Problems:

- hard to generate
- hard to edit programmatically
- no real-time pipeline

The deeper insight

There are **two fundamentally different document paradigms**:

1. Structure-first (Notion-like)

data → render



2. Layout-first (Word-like)

content + style → layout → render



Where Word actually shines

Word is unbeatable when you need:

- final polished documents
- print-ready formatting
- contracts, reports, academic papers

Where it breaks for you

You are trying to:

- generate docs programmatically
- iterate quickly
- collaborate live
- integrate with LLMs

Word is **not designed for this**.

Blunt takeaway

- Word is a **publishing engine**
- Notion is a **knowledge system**

You're currently stuck because:

you're trying to use publishing tools in a generation workflow

The clean mental model for your system

LLM → structured doc → collaborative system → (optional) export → Word 

Word should be the **last step**, not the working environment.

If you want, I can go one level deeper and show:

- exactly how Pandoc maps Markdown → Word XML
- and why that pipeline feels so painful (it'll click instantly)

    ...

how does word keep track of comments/layout?

Good—this gets into the *real machinery* of Word.

At a high level:

Word tracks comments and layout using two separate systems:

- "Comments = anchored metadata linked to text ranges"
- "Layout = computed dynamically from content + styles + page rules"

These are very different internally.

1. How Word tracks comments

Core idea

A comment is **not inside the text**.

It's stored separately and linked to a **range of characters**.

Structure (simplified)

Inside the .docx (ZIP), you have:

- document.xml → main content
 - comments.xml → all comments
-

In document.xml (anchor markers)

```
<w:p>
  <w:r><w:t>This is </w:t></w:r>

  <w:commentRangeStart w:id="0"/>
  <w:r><w:t>important</w:t></w:r>
  <w:commentRangeEnd w:id="0"/>

  <w:r>
    <w:commentReference w:id="0"/>
  </w:r>
</w:p>
```

👉 This says:

- the word **"important"** is linked to comment id=0
-

In comments.xml

```
<w:comment w:id="0" w:author="Igor">
  <w:p>
    <w:r><w:t>Consider rewording this</w:t></w:r>
  </w:p>
</w:comment>
```

Key properties

- Comments are **range-based**, not block-based
- They survive formatting changes (mostly)
- They can attach to:

- words
 - sentences
 - paragraphs
-

Why this matters

This is very different from Notion:

- Notion → comments attached to blocks
- Word → comments attached to **character ranges**

👉 Word is much more precise for editing.

2. How Word tracks "track changes"

Closely related to comments.

Instead of replacing text, Word wraps it:

```
<w:del>
  <w:r><w:t>old text</w:t></w:r>
</w:del>

<w:ins>
  <w:r><w:t>new text</w:t></w:r>
</w:ins>
```

👉 The document contains BOTH versions.

The UI decides:

- show both (review mode)
 - show final
 - show original
-

3. How Word handles layout (this is the big one)

Core idea

Word does NOT store layout.
It computes layout every time.

Inputs to layout engine

Word takes:

1. Content

- paragraphs (w:p)
 - runs (w:r)
 - tables, images
-

2. Styles (styles.xml)

- font size
 - spacing
 - indentation
-

3. Section settings

```
<w:sectPr>
  <w:pgSz w:w="12240" w:h="15840"/> <!-- page size -->
  <w:pgMar w:top="1440" w:left="1440"/> <!-- margins -->
</w:sectPr>
```

4. Layout rules

- line wrapping
 - justification
 - widow/orphan control
 - page breaks
-

Then Word computes:

content + styles + rules → layout engine → pages



Example: line wrapping

Word decides:

- how many words fit in a line
- where to break lines
- where pages split

None of that is stored explicitly.

4. Why layout is “fragile”

Because it’s computed, small changes cause big effects:

- change one word →
- line shifts →
- page break changes →
- everything after moves

👉 That’s why Word docs can “jump around”

5. Anchoring for objects (images, etc.)

Images are positioned using anchors:

```
<w:drawing>
  <wp:anchor>
    <!-- position relative to paragraph/page -->
  </wp:anchor>
</w:drawing>
```

Options:

- inline (flows with text)
- floating (absolute-ish position)

6. Why this system is powerful

Word can:

- precisely comment on text
- track edits legally
- produce consistent print layouts
- support complex documents

7. Why it's painful for your use case

Because:

1. Everything is tied to text positions

- not semantic structure

2. Layout is implicit

- not stored as data
- hard to control programmatically

3. XML is extremely verbose

- generating it reliably is hard

4. Round-tripping is messy

- LLM → docx → edit → re-import = pain

Clean comparison to your problem

Word model

text → annotate ranges → compute layout → render



What you want

structured data → render → collaborate → export



The key insight

Word answers:

"How do we produce perfect documents?"

You are trying to solve:

"How do humans + LLMs co-create documents?"

Those are different problems.

Blunt takeaway

- Comments → anchored to character ranges via IDs
- Layout → fully computed from styles + rules (not stored)

And the reason this feels hard in your workflow is:

Word is optimized for **editing finished documents**, not **generating them programmatically**

If you want, I can show:

- a minimal example of generating a `.docx` correctly
- and exactly where Pandoc helps vs where it breaks
(which will clarify why your current pipeline feels so heavy)



so why cant an llm just write these instead?

It *can*. That's not the real blocker.

The reason your workflow still feels painful is:

Writing Word XML is easy. Writing *correct, stable, collaborative, layout-safe Word documents* is hard.

Let's break that down clearly.

1. Yes — an LLM can generate .docx

A model can absolutely output:

- valid Office Open XML XML
- zipped into .docx
- with paragraphs, headings, even comments

And tools like Pandoc already prove this works.

So capability = **not the issue**

2. Where it breaks (this is the real reason)

A. Word documents are not just "text → format"

They are:

text + styles + relationships + layout rules + references



That means:

- multiple XML files must stay in sync
- IDs must match across files
- references must be valid

Example:

- comment in comments.xml
- must match w:id in document.xml

👉 One mismatch → broken doc

B. Layout is *implicit*, not declared

This is the killer.

LLM writes:

"This paragraph should look nice"



But Word needs:

font size + spacing + margins + page rules → layout engine decides re

👉 You cannot directly control:

- where page breaks happen
- how text wraps
- how layout shifts after edits

So:

- output is **non-deterministic**
- small edits break formatting

C. It's extremely verbose and fragile

Compare:

Markdown

```
# Title
```

Word XML

```
<w:p>
  <w:pPr><w:pStyle w:val="Heading1"/></w:pPr>
  <w:r><w:t>Title</w:t></w:r>
</w:p>
```


Now scale that to:

- tables
- images
- comments
- track changes

👉 It explodes in complexity

D. No clean abstraction for LLMs

LLMs work best when:

- structure is simple
- rules are consistent
- outputs are composable

Word is:

- deeply nested
- stateful
- cross-referenced
- layout-dependent

👉 Bad fit for generation

E. Round-tripping is the real nightmare

Even if the LLM generates a perfect doc:

1. Teammate edits in Word
2. You want to re-process it with LLM

Now you must:

- parse XML
- reconstruct meaning
- deal with layout artifacts

👉 This is where pipelines collapse

3. Compare to a “good” LLM-friendly system

Word approach

LLM → XML → layout engine → doc



Problems:

- fragile
- hard to edit programmatically
- not composable

Notion-style approach

LLM → structured blocks → render



Advantages:

- deterministic
- editable
- composable
- API-friendly

4. The deeper reason (this is the key insight)

Word encodes **presentation-first** documents

LLMs naturally generate **structure-first** content

Mismatch.

Word thinks:

"How should this look on a page?"

LLM thinks:

"What is the structure and meaning?"

5. Why Pandoc exists (and why you're using it)

Because nobody wants to deal with Word XML directly.

Pandoc acts as:

clean format (Markdown) → messy format (Word XML)



So your current workflow:

LLM → Markdown → Pandoc → Word



is actually the *industry workaround*

6. So why not skip Markdown?

You *can*, but then you must:

- design a schema
- ensure validity
- build renderer/exporter
- handle edits + re-import

At that point you're basically:

building your own Notion / Google Docs layer

7. The honest answer

Can LLMs write Word docs directly?

Yes.

Should they be your primary interface?

No.

Because:

- too fragile
- too complex

- poor for iteration
 - bad for collaboration loops
-

8. The real bottleneck (your intuition is close)

You're thinking:

"LLM already can generate structure, so why isn't this solved?"

Answer:

Because generating structure $\neq$ managing a living document system

The missing piece is:

- persistence
 - collaboration
 - editing
 - versioning
 - stability after humans touch it
-

Final blunt takeaway

- LLMs *can* write Word docs
- But Word is a **terrible target format for generation workflows**

It's like:

generating compiled binaries instead of source code

What you should do instead

Use Word only as:

final export layer



Keep your pipeline in:

LLM → structured representation → collaborative system → export to Word 

If you want, I can show you a **minimal end-to-end setup** where:

- Claude outputs structured JSON
- you render it in a simple web UI
- and export to Word cleanly

That's the cleanest version of what you're trying to build.



so do you understand what I want to do?

Yes—and I'm going to say it back bluntly to make sure we're aligned.

What you're actually trying to do

You want:

Claude (or any LLM) to generate documents that are immediately readable, shareable, and editable by teammates — without requiring Markdown tools, Pandoc, or technical workflows.

More concretely:

Your ideal workflow

You → prompt Claude → doc appears → share link → teammate opens → read 

NOT:

Claude → .md → VS Code → Pandoc → .docx → open → edit → break pipeline 

What you *don't* want (this is important)

- ❌ Teammates needing dev tools (VS Code)
- ❌ Markdown as the "user-facing format"
- ❌ Conversion pipelines (Pandoc, XML, etc.)
- ❌ Multi-step workflows
- ❌ Fragile formatting

What you *do* want

1. Immediate readability

- opens like Google Docs or Notion
- not raw text

2. Collaboration

- comments
- edits
- shared access

3. LLM-native generation

- Claude writes directly into the doc
- no conversion layer

4. Minimal friction

- no tooling knowledge required for teammates
-

What's subtly important (and easy to miss)

You're not just solving:

"Markdown is annoying"

You're solving:

"There is no clean interface between LLM-generated content and human collaboration."

Why your current pain exists

Because your system is:

LLM → developer format (Markdown) → human consumption



But Markdown is:

- great for storage
- bad for non-technical users

What you're really asking (at a deeper level)

You're asking:

"What is the *right medium* for LLM-generated documents?"

And you've discovered:

- Markdown → too raw
- Word → too heavy
- Notion → close but limited

That's exactly the right insight.

My honest read of your intent

Given your background (ML + systems thinking), you're not just trying to *use* a tool.

You're trying to figure out:

Is there a better paradigm for document workflows in the LLM era?

And the answer is:

yes — but it's not fully solved yet.

Where you are in the problem space

You've already identified:

- LLMs can generate structure ✓
- conversion pipelines are painful ✓
- current tools are mismatched ✓

That puts you at:

"pre-product insight stage"

This is exactly where good products start.

One sentence summary

You want:

LLM → native collaborative document (no markdown, no conversion, no friction)

If I had to push you slightly

Right now you're circling between:

- Notion
- Word
- Markdown

But the real question you should answer is:

Where should the "source of truth" for documents live in an LLM-first workflow?

Once you answer that, everything else becomes obvious.

If you want, next step we can do is:

- define **exact requirements for your ideal system**
- map **why every existing tool fails those requirements**
- and identify **what the simplest MVP would be**

That would turn this from exploration → something you could actually build or validate.



no. I want to create a new markdown format, lets call it mdxx. This will have all information that is in xml usually but just in markdown, then my editor will render everything.

Imagine from comments to fonts, to layout, everything in markdown. At the bottom, there will be a "skill" about how to add on/edit the markdown for the agents to have since they are not trained on this format for now.

Thought for 13s >

Yes. That is much clearer, and it is a stronger idea than “make Markdown easier to share.”

What you want is not a better editor for Markdown. You want a **new source format**:

mdxx = human-writable plain text that carries both semantic structure and rich document metadata, then a custom editor renders it fully.

That means:

- comments live in the source
- fonts and styles live in the source
- layout rules live in the source
- agents can edit the source directly
- the editor is the renderer, not Word
- a trailing “skill” or schema guide teaches agents how to write valid mdxx

That is a coherent idea.

The good news

This is absolutely a real design space. Existing systems already point in this direction.

CommonMark standardized basic Markdown because plain Markdown was too ambiguous. CommonMark Sp... +1

Pandoc extends Markdown with many format-specific extensions. Pandoc

MyST goes further and adds directives and roles so Markdown can represent much richer document structures. mystmd.org +2

Typst is even closer in spirit to what you want: a text-based authoring language where layout and styling are first-class concepts rather than an afterthought. Its docs explicitly expose layout and styling primitives like pages, grids, columns, alignment, and style rules. Typst +1

So the core thesis is valid:

plain text can absolutely be the source of truth for rich documents.

The hard truth

If you include “everything in Word XML, but in Markdown,” you are no longer making a Markdown format in the normal sense. You are designing a **document programming language**.

That is not bad. It just changes the problem.

The real risk is this:

if mdxx tries to encode every possible Word-style visual detail directly, it will become ugly, brittle, and agent-hostile.

That is where these projects usually die. They become:

- too verbose for humans
- too underspecified for renderers
- too stateful for agents
- too layout-dependent for collaboration

So the right question is not “can mdxx contain everything?”

The right question is:

What is the minimum rich document model that gives you 90% of the value without recreating Word?

My recommendation

Do **not** make mdxx “Markdown plus raw layout micromanagement everywhere.”

Make it 3-layered.

Layer 1: semantic document body

This stays Markdown-like.

Example:

```
# Proposal
```

```
This is a paragraph with a [comment:id=c1 span="paragraph-1"]review note[/  
]
```

```
## Goals
```

- fast collaboration
- agent-editable source
- rich rendering

Layer 2: directives / attributes / metadata

This is where mdxx gets richer, closer to MyST or Pandoc extensions. mystmd.org +2

Example:

```
::page  
size: letter  
margin: 1in  
columns: 1  
font-family: Inter  
font-size: 11pt  
::  
  
::comment{id="c1" author="Igor" anchor="paragraph-1"}
```

Tighten this section.
::

Layer 3: renderer-specific style sheet

Do not force every document line to carry every layout detail inline.

Example:

```
::styles
heading-1:
  font-size: 24pt
  weight: 700

paragraph:
  spacing-after: 10pt
::
```

That separation matters a lot. It keeps the body readable and gives the renderer enough structure.

The best design principle

Make mdxx source express:

- **structure**
- **annotations**
- **intent**
- **stable anchors**

And let the renderer compute most layout.

Do not make the source express:

- exact line wraps
- exact page coordinates for normal text
- fragile presentation state

That is the same trap Word falls into.

About comments

Comments are actually a strong fit for mdxx.

But I would not encode them as “hidden text markers” the way Word effectively uses range anchors. Word stores comments separately and links them to text ranges by IDs.

CommonMark Sp... +1

- Pandoc Markdown for extension mindset [Pandoc](#)
- MyST for directive/role-based richness [mystmd.org](#) +2
- Typst for “text source controls layout” philosophy [Typst](#) +1

If you ignore those, you’ll likely reinvent them badly. If you learn from them, mdxx could be sharp.

My blunt assessment

This idea makes sense if you narrow it to:

“Markdown-shaped syntax for a structured rich-document model with stable anchors, annotations, and stylesheet-driven rendering.”

It becomes a bad idea if you define it as:

“all the information in Word XML, but written in Markdown.”

That second version is too much. It will bloat immediately.

The right MVP

Your first mdxx should probably support only:

- headings
- paragraphs
- lists
- tables
- code blocks
- callouts
- comments
- document metadata
- named styles
- page/section layout
- stable anchors

Not:

- everything Word can do

That is enough to prove the concept.

The one-sentence verdict

Yes, I understand it now, and the principled version of your idea is good: build a **Markdown-shaped rich document language**, not a Markdown clone that tries to swallow raw XML complexity.

I can help you draft an actual `mdxx v0.1` syntax spec next.

    ...    Sources

how can i export this conversation?

Here are the **clean, practical ways** to export this conversation depending on how polished you want it:

1. Fastest (good enough): Copy → paste into doc

- Select all text in the chat
- Paste into:
 - Google Docs
 - Notion

Then:

- clean formatting
- share with teammate

👉 Best if you just need it quickly

2. Clean export (recommended for you)

Option A: Print → PDF

- Cmd + P (Mac)
- Destination: **Save as PDF**

Pros:

- preserves structure well

- easy to share
- no tooling needed

Cons:

- not editable

Option B: Export via ChatGPT UI (if available)

Some versions have:

- "Share" → link
- or "Export" options

If you see:

- "Share conversation" → use it

3. Structured export (best for your mdxx idea)

Since you're thinking systemically, do this:

Ask me:

"Convert this conversation into structured markdown / JSON / mdxx draft"

I can turn it into something like:

JSON

```
{
  "sections": [
    {"title": "Problem", "content": "..."},
    {"title": "Notion vs Word", "content": "..."}
  ]
}
```



or even:

Markdown

```
# mdxx draft
```

```
:::section
```



title: Problem

...

👉 This is actually the **most valuable** path for you